



DEPARTMENT OF SOFTWARE ENGINEERING

COURSE INSTRUCTOR: ENGR. AHMED KHAWAJA

PROJECT

SUBMITTED BY: Muhammad Abdullah Ashir **ROLL NUMBER:** 2023-SE-39

SUBMITTED BY: Syed Muhammad Abdullah Gillani **ROLL NUMBER:** 2023-SE-24

COURSE TITLE: Deep Learning

COURSE CODE: SE-3210

SESSION: 2023-27

Table of Contents

1. Overview	3
Key points	3
2. Version History and Results.....	4
3. Source Models	4
4. Notebook Structure	5
5. Configuration.....	5
6. Model Pipelines	6
6.1 Perch + ProtoSSM + ResidualSSM (Model_21 / Model_22).....	6
6.2 Power Optimization pipeline (Model_51 / 52 / 73 / 74).....	7
6.3 Distilled SED (Model_1).....	7
7. Blending and Post-Processing	8
7.1 Taxonomic smoothing	8
7.2 Safety checks	8
8. Saved Run and Observations.....	9
9. Environment and Inputs.....	9
10. Glossary	10

TECHNICAL REPORT

BirdCLEF+ 2026 – EoS.9

Ensemble notebook for acoustic species identification in the Pantanal

Source file	birdclef-2026-eos-9.ipynb (Kaggle notebook, 34 cells, Python 3 kernel)
Competition	BirdCLEF+ 2026 (Kaggle) – 234-class soundscape species prediction
Active configuration	Model_22 + Model_51 + Model_74, direct weighted blend, taxonomic smoothing
Best recorded public LB	0.950 (v.12, and earlier v.10/v.11)
Report prepared	September 20, 2026

1. Overview

EoS.9 is a consolidation notebook for the BirdCLEF+ 2026 competition. It does not introduce a new model. Instead it bundles the code of several strong public solutions, runs the ones selected in a small configuration dictionary, and blends their predictions into a single `submission.csv`.

The task is to predict, for every 5-second window of a 60-second soundscape recording (12 windows per file), the probability that each of 234 classes is vocalising. The submission therefore has one row per window and 235 columns: a `row_id` followed by 234 class-probability columns.

The notebook's own contribution is the last layer of the pipeline: how the models are weighted, which blending scheme is used, and a taxonomy-based smoothing step applied after blending. The authors also record the leaderboard (LB) result of each version, so the notebook doubles as an experiment log.

Key points

- **Active models:** Model_22, Model_51 and Model_74. The remaining model cells (Model_1, 21, 52, 73) are kept in the file but only run if their name appears in the configuration.
- **Blend weights (v.13):** 0.012 / 0.021 / 0.967, dominated by Model_74.
- **Post-processing:** taxonomic smoothing with genus $\alpha = 0.15$ and class $\alpha = 0.05$.
- **Result:** public LB 0.950 for v.12; v.13 was still pending when the notebook was saved.

2. Version History and Results

The notebook header records each submitted version with its blend type, weights and public leaderboard score. All versions use an internal SED / ProtoSSM rank blend of 0.60 / 0.40 for the third model (Model_74 or its equivalent).

Version	Models	Blend	Weights	LB
v.7.prev	21 + 52 + 74	division_attention	0.014 + 0.021 + 0.965	0.949
v.8.prev	21 + 52 + 74	division_attention + TAX_SMOOTHING	0.014 + 0.021 + 0.965	0.949
v.10.prev	22 + 51 + 74	direct + TAX_SMOOTHING	0.022 + 0.0085 + 0.9695	0.950
v.11.prev	22 + 74 + 51	direct + TAX_SMOOTHING	0.022 + 0.0085 + 0.9695	0.950
v.12	22 + 51 + 74	direct + TAX_SMOOTHING	0.020 + 0.013 + 0.967	0.950
v.13	22 + 51 + 74	direct + TAX_SMOOTHING	0.012 + 0.021 + 0.967	pending

The authors give the ordering of the earlier versions as $v.8 < v.7 < v.11 < v.10$. The component models scored 0.928 (Model_22), 0.949 (Model_51) and 0.949 (Model_74) on their own public notebooks. Adding taxonomic smoothing on its own (v.7 to v.8) left the score at 0.949. The move to 0.950 came with the switch to Models 22 + 51 and the direct blend in v.10.

The authors also pose an open question in the notebook: whether the drop seen between versions comes from the rank-weighting schemes, from the additional small weighting scheme (a difference of about 0.0045), or from a bug. They note that any answer would be useful before the end of the competition.

3. Source Models

Each model below comes from a public Kaggle notebook by a different author. The notebook copies their code into a gated cell (for example, if 'Model_22' in `_ensemble_models`), so a model's code is executed only when it is listed in the configuration.

Model IDs	Source notebook	Author	Public LB	In v.13 blend
Model_1 (11, 12)	BC2026 Distilled-SED	Tucker Arrants	0.917	No (its SED folds are reused)
Model_21, Model_22	Bird26.REPRODUCE.Perch+ProtoSSM+ResSSM.INF/TRAIN	yukiZ	0.928	Model_22
Model_51, Model_52	birdclef 2026 exp019 eos4 rank power 06	Derek	0.949	Model_51
Model_73, Model_74	v6_0949_replay	Yaroslav Kholmirezayev	0.949	Model_74

The header table also lists Model_11 and Model_12 as variants of Model_1. This file contains code cells for Model_1, 21, 22, 51, 52, 73 and 74 only.

4. Notebook Structure

The notebook has 34 cells. The table lists them by 0-based index.

Cells	Section	Content
0 – 2	Introduction (markdown)	Context, links, version tables, thoughts on earlier runs
3 – 5	preparing_parameters	The solutions dictionary and the lists derived from it (model names, files, weights, xSED, LB)
6 – 7	Model_1	Distilled SED: training and inference code (about 1,240 lines)
8 – 11	Model_21, Model_22	Perch + ProtoSSM + ResidualSSM pipeline (about 3,400 lines each)
12 – 17	Model_51, Model_52	Derek's notes (cells 12–13) and the Power Optimization pipeline (cells 14–17)
18 – 23	Model_73, Model_74	Yaroslav's notes (cells 18–19) and the pipeline (cells 20–23)
24 – 26	division_attention check	Counts the columns of the two column groups used by the alternative blend
27 – 31	blend	Blend functions, taxonomic smoothing, and selection of the active blend
32 – 33	Submit	Runs the selected blend and writes submission.csv

5. Configuration

All behaviour is controlled by a single dictionary defined in the preparing_parameters section:

```
solutions = {
  'type_add' : 'TAX_SMOOTHING',
  'task1'    : 'run SED once',
  'task2'    : {'save PSSM for Model_51 as':'subm_51p.csv'},
  'Models'   : [
    {'Model':'Model_22','subm':'subm_22.csv', 'weight':0.012,'xSED':[], 'LB':'0.928'},
    {'Model':'Model_51','subm':'subm_51p.csv', 'weight':0.021,'xSED':[], 'LB':'0.949'},
    {'Model':'Model_74','subm':'subm_74.csv', 'weight':0.967,'xSED':[0.60,0.40], 'LB':'0.949'}
  ]
}
```

Key	Meaning
type_add	Selects the blend function. TAX_SMOOTHING runs the direct weighted sum, then applies taxonomic smoothing.
task1	“run SED once”: the Distilled SED inference is computed once and reused rather than repeated for each model.
task2	Model_51 saves its ProtoSSM (PSSM) predictions as subm_51p.csv, so that its file matches the configuration.
weight	Weight of each model in the final direct blend. The three weights sum to 1.0.
xSED	[ProtoSSM weight, SED weight] for a rank blend inside a model. Empty for Models 22 and 51; [0.60, 0.40] for Model_74.
LB	Public leaderboard score of the source notebook, used for logging only.

6. Model Pipelines

Figure 1 shows how the active pieces fit together. The models share one main ingredient: embeddings from Google's Perch v2 bioacoustic model, which the ProtoSSM-family networks then process as a sequence.

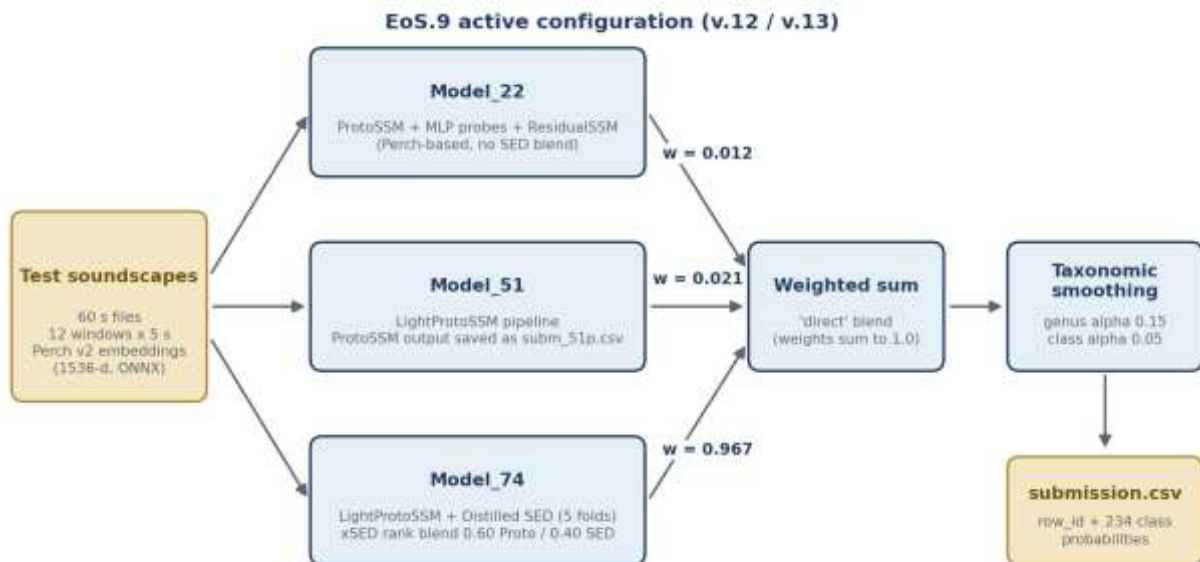


Figure 1: Active blend configuration of EoS.9

6.1 Perch + ProtoSSM + ResidualSSM (Model_21 / Model_22)

This pipeline works on sequences of Perch embeddings rather than on raw audio. Its stages, as seen in the code and the saved run log, are:

- Extract 1536-dimensional Perch v2 embeddings (ONNX runtime) for every 5-second window, and cache them.
- Map Perch outputs to the competition classes: 203 of 234 classes have a Perch logit, 31 do not (3 receive frog proxy targets, 28 have no signal).
- Build site and hour-of-day prior tables from the labelled soundscapes and fuse them with the base scores.
- Train ProtoSSM v4: a bidirectional state-space sequence model with temporal cross-attention and prototype-based classification (d_model 256, 3 SSM layers, d_state 16, 4 attention heads), using mixup, focal loss and SWA.
- Train class-wise embedding probes on PCA-reduced embeddings, then a ResidualSSM that learns additive corrections to the first-pass predictions.
- Apply per-taxon temperature scaling (Aves 1.10, Amphibia/Insecta 0.95), shift-based test-time augmentation, rank-aware scaling and delta smoothing.

The training data in the saved run consist of 59 fully labelled soundscape files (708 windows). In the log the MLP probes use PCA with 128 components (about 90% of the variance).

6.2 Power Optimization pipeline (Model_51 / 52 / 73 / 74)

These four cells share one code base, described by the authors as a “Power Optimization” pipeline that scored 0.948–0.949 publicly. It extends the pipeline above with a Distilled SED branch and heavier post-processing:

- LightProtoSSM with cross-attention (d_model 128, d_state 16, 2 SSM layers, 2 attention heads), trained on the labelled soundscape embeddings and applied with shift TTA (shifts 0, ± 1 , ± 2).
- PCA to 64 components (81.5% of the variance retained) and 58 MLP probes on sequence-aware features.
- ResidualSSM with a grid search over the correction weight. In the saved log the best value is 0.35, with an out-of-fold macro-AUC of 0.9937.
- Post-processing chain: prior fusion (lambda_prior), file-level confidence scaling (top-k 2, power 0.4), rank-aware scaling, adaptive delta smoothing and per-class threshold calibration (31 classes calibrated, mean threshold 0.464).
- Distilled SED inference with five ONNX folds (sed_fold0 to sed_fold4), then a rank-based xSED blend of ProtoSSM and SED (0.60 / 0.40).
- Final clean-up: sonotype mirroring across 10 columns, adaptive thresholding for 44 rare species, a proto_cont rule that protects confident ProtoSSM detections, and a diagnostic check (235 columns, probabilities within [0, 1], no missing values, no duplicated row_id).

The four cells differ only in a handful of scalar settings. These are the values found in the code:

Setting	Model_51	Model_52	Model_73	Model_74
lambda_prior (test / calibration)	0.65 / 0.65	0.5 / 0.5	0.75 / 0.65	0.75 / 0.65
rank_aware_scaling power	0.65	0.6	0.6	0.6
proto_cont: rank_proto >	0.75	0.75	0.75	0.77
proto_cont: p_sed <	0.12	0.12	0.12	0.14
xSED (ProtoSSM / SED)	0.60 / 0.40	0.60 / 0.40	0.60 / 0.40	0.60 / 0.40

Model_74 carries the change suggested in the notebook header by the contributor “swordsman”: the proto_cont thresholds move from (rank_proto > 0.75, p_sed < 0.12) to (0.77, 0.14), which reportedly gave a slightly higher score. The old line is left in the code as a comment. The header notes that the change can be applied to Model_52 and Model_74 together or separately.

6.3 Distilled SED (Model_1)

Model_1 is a frame-wise Sound Event Detection network trained with teacher–student distillation. It is not in the active blend, but its five trained folds are loaded by the Power Optimization cells to provide the SED half of the xSED blend.

- **Input:** mel spectrograms of 5-second clips at 32 kHz (256 mels, n_fft 2048, hop 512, 20–16,000 Hz).
- **Backbone:** tf_efficientnet_b0.ns_jft_in1k, with learnable Generalized-Mean (GeM) pooling over the frequency axis and an attention bottleneck that turns frame logits into clip logits.
- **Distillation:** an extra head projects the features into the 1536-d Perch v2 embedding space (MSE loss, weight 1.0).

- **Training:** 5 folds, 25 epochs, batch size 64, sources weighted 0.9 focal recordings / 0.1 labelled soundscapes.
- **Augmentation:** gain jitter (± 6 dB), noise injection (SNR 10–30 dB), SpecAugment, focal–focal mixup and focal–soundscape mixup, with rare species upsampled to at least 20 samples per fold.

7. Blending and Post-Processing

The blend section (cells 27–31) defines the functions that combine the per-model submission files. The function that is used is selected by `type_add`.

type_add	Function	How it combines the models
direct	<code>direct_add2 / direct_add3</code>	Plain weighted sum of probabilities: $\sum \text{weight} \times \text{prediction}$.
rank.1	<code>rank_1_add2 / rank_1_add3</code>	Converts each model's probabilities to per-column percentile ranks, then takes a weighted sum of ranks.
division_attention	<code>division_attention</code>	Uses one weight set for the first 117 class columns and a slightly different set for the rest (0.014/0.021/0.965 vs 0.0137/0.0213/0.965).
TAX_SMOOTHING	<code>f_TAX_SMOOTHING_POSTPROC</code>	Runs the direct blend, then applies taxonomic smoothing (active setting).

7.1 Taxonomic smoothing

After the weighted sum, each class probability is pulled slightly toward the mean of its taxonomic group for the same window. Genus is taken from the first word of the scientific name in `taxonomy.csv`, and class comes from the `class_name` column. Only groups with more than one member are smoothed. For a window and a group G :

$$p_{\text{new}}[c] = (1 - \alpha) * p[c] + \alpha * \text{mean}(p[G]) \quad \text{for each class } c \text{ in } G$$

Genus smoothing first ($\alpha = 0.15$), then class smoothing ($\alpha = 0.05$).

The saved run reports 29 multi-member genus groups and 4 multi-member class groups. The intuition is that a call detected for one species is weak evidence for its close relatives, which may help rare species whose scores are noisy. The step was taken from the EoS.8 notebook, where a contributor reportedly obtained a leaderboard score of 0.950 with it.

7.2 Safety checks

A helper (`_read_submission_checked`) verifies that a submission file has a unique `row_id`, finite values and probabilities in $[0, 1]$. A second helper (`direct_addsafe`) additionally normalises the weights and checks that `row_ids` and columns match across files. In this version, `direct_addsafe` is defined for two models only and is not used by the active blend.

8. Saved Run and Observations

The notebook was saved after a run in which the hidden test set was not available, so the model cells fell back to a dry run on 20 training files. This affects what the stored outputs show, and it is worth knowing before reading them:

- **Dry-run submissions.** Model_74's output has 3 rows (test placeholders, 235 columns), while the Model_22 and Model_51 outputs shown in the final cell use training-file row_ids. The stored values are therefore not real predictions.
- **Row mismatch in the final blend.** The final cell prints a blended frame of shape (243, 234) with mean=nan. This is consistent with adding DataFrames whose row_ids do not overlap (240 training rows + 3 test rows). On the real test set all three files share the same row_ids, so this should not occur, but the direct blend does not check for it.
- **Run time.** The ProtoSSM part of the dry run took about 2.8–2.9 minutes, and the PSSM training itself only 7–15 seconds.

A few smaller points found while reading the code:

- Comments in the Model_51 cell still describe an earlier experiment (lambda_prior 0.5, power 0.6), while the code uses 0.65 and 0.65. The code values are the ones in effect.
- The author excerpt for Model_74 describes a larger ProtoSSM (d_model 320, 4 layers, 8 heads), but the cell instantiates LightProtoSSM with d_model 128, 2 layers and 2 heads. The excerpt also mentions a ResidualSSM correction weight of 0.10, whereas the saved log selects 0.35.
- The division_attention check in cell 25 splits the 234 columns by the substring “son” (209 / 25 columns), while division_attention() itself splits at column index 117. The two are not the same rule, and this blend is not part of the active configuration.

9. Environment and Inputs

Item	Detail
Runtime	Python 3.12 on Kaggle, CPU only (a CUDA initialisation warning appears in the log)
Libraries	TensorFlow 2.20, PyTorch 2.9 (CPU), ONNX Runtime 1.24.4 installed from a bundled wheel, scikit-learn, pandas, NumPy
Competition data	sample_submission.csv, taxonomy.csv, train soundscape labels, soundscape audio
Perch v2	perch_v2_no_dft.onnx, plus a precomputed embedding cache (perch-meta dataset)
SED weights	Five ONNX folds from the bc2026-distilled-sed-public dataset
Outputs	Per-model CSV files (subm_22.csv, subm_51p.csv, subm_74.csv) and the final submission.csv

Model code loads its inputs from /kaggle/input paths, so the notebook is meant to run on Kaggle with the listed datasets attached and no internet access at scoring time.

10. Glossary

Term	Meaning
BirdCLEF+	Kaggle competition series on identifying species from soundscape audio
LB	Leaderboard score (public test split)
Perch v2	Google’s bioacoustic embedding model, used as a frozen feature extractor
SED	Sound Event Detection: frame-wise prediction of sound events
ProtoSSM	State-space sequence model that classifies by similarity to learned class prototypes
xSED	Weights of the ProtoSSM / SED rank blend inside a model
OOF	Out-of-fold: predictions on data held out during cross-validation
TTA	Test-time augmentation (here, circular shifts of the window sequence)
GeM	Generalized-mean pooling with a learnable exponent
PCA	Principal component analysis, used to reduce embedding dimensionality